Data Structures and Algorithms

Lecture 3 - Queues

Dept. of ICT, Faculty of Technology- UOR, 2026

Queues

A Queue is a data structure that is similar to a Stack.

In a Queue the first item inserted is the first item to be removed (FIFO).

- In stack last item inserted is the first to be removed (LIFO).

There are various queues quietly doing their job in a computer's operating system.

- printer queue.
- stores keystroke data as you type at the keyboard.
- pipeline.

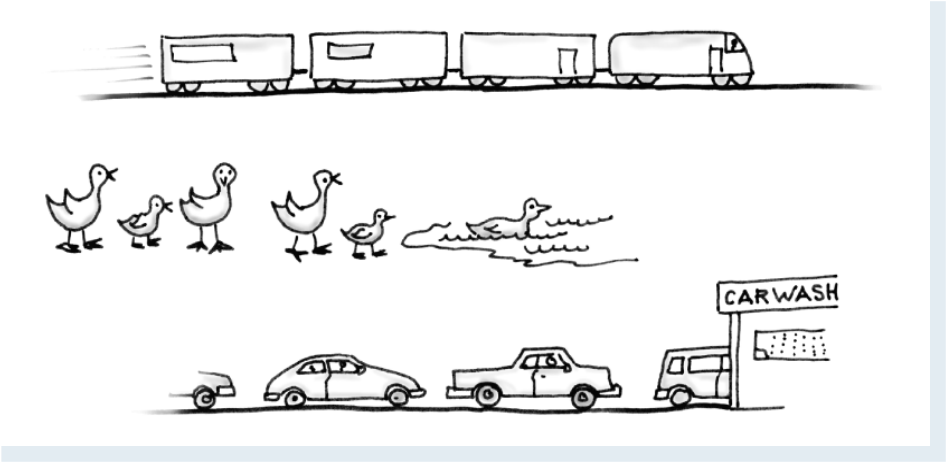


Queues

Stores a set of elements in a particular order

Queue principle: FIRST IN FIRST OUT (FIFO)

It means: the first element inserted is the first one to be removed



Queues

Queues are lists of elements with restricted insertion and deletion operations.

Insertions are made at the Rear end and Deletions are made at the Front end.

Elements are Accessed from Front end of the Queue.

Insertions and Deletions are restricted from the Middle of a Queue data structure.

Elements are inserted and removed according to the First-In-First-Out (FIFO) principle.

In Brief ……

A First in First Out (FIFO) list.

Like queuing at the bank or the bus stop.

First in the queue is first to be served by the next available bank teller.

First in the queue at the bus stop is first to get on the bus when it arrives.

Operations on a Queue

Operation	Description
dequeue	Removes an element from the front of the queue
enqueue	Adds an element to the rear of the queue
First (peek)	Examines the element at the front of the queue
isEmpty	Determines whether the queue is empty
size	Determines the number of elements in the queue
toString	Returns a string representation of the queue

Examples.....

In real world Queue is defined as a waiting line.

Ex:

- At Grocery Stores
- At Banks
- Aeroplanes at Airports
- Internet- Data packet



In computer systems.

Ex:

- Job Queue - multi-user systems processes waiting for the CPU
- Print Queue - processes waiting for the printer

In most computer installations, for example, one printer is connected to several different machines so that more than one user can submit printing jobs to the same printer, it maintains a queue.

The number of elements in the queue is equal to the value of

rear - front + 1

Real World Applications of Queue Data Structures

Computer Simulation Programs

Printer Spooling (Simultaneous Peripheral Operations Online)

Computer and Video Games

Processor Scheduling Algorithm

Queue of Packets in Data Communication

Various Queues

Normal queue (FIFO)

Circular Queue (Normal Queue)

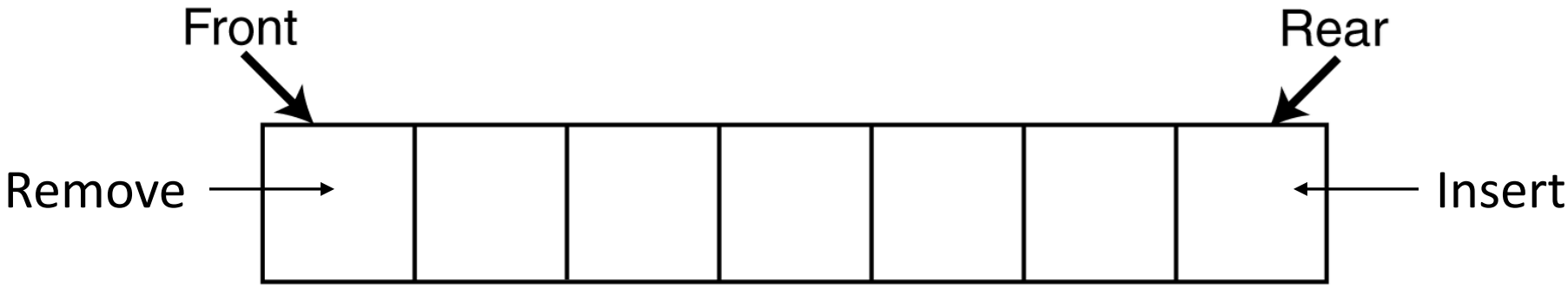
Double-ended Queue (Deque)

Priority Queue

Queue Operations

Queues having a front and a rear.

Items may be deleted at one end (called front of the queue) and into which items may be inserted at the other end (called the rear of the queue)



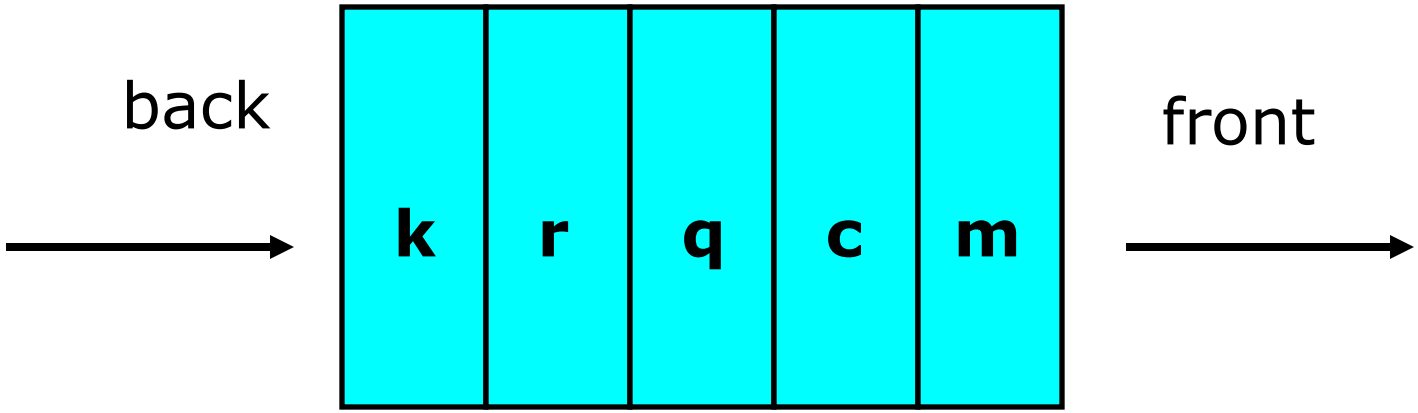
Queue Operations

The two primary queue operations are ***enqueuing*** and ***dequeuing*.**

enqueue -insert an element at the rear of a queue.

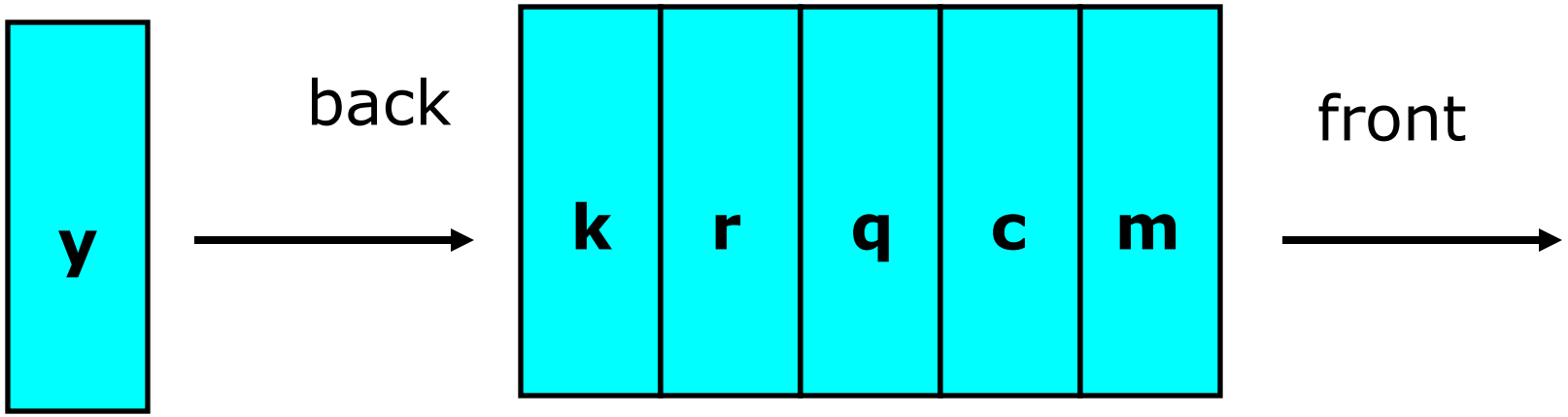
dequeue - remove an element from the front of a queue.

Queues are FIFO



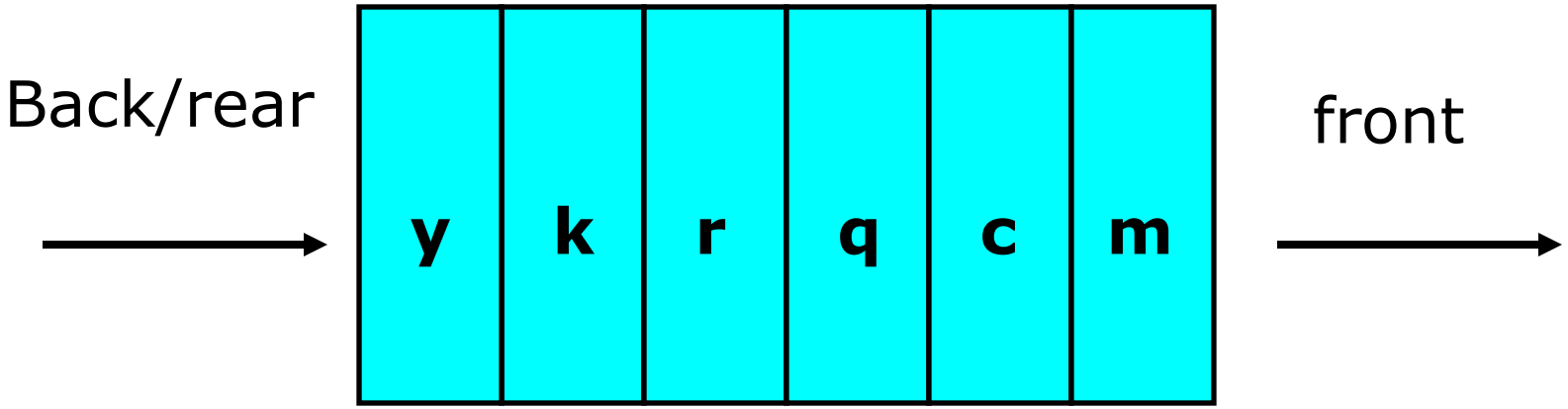
Queues are FIFO

Enqueue operation:



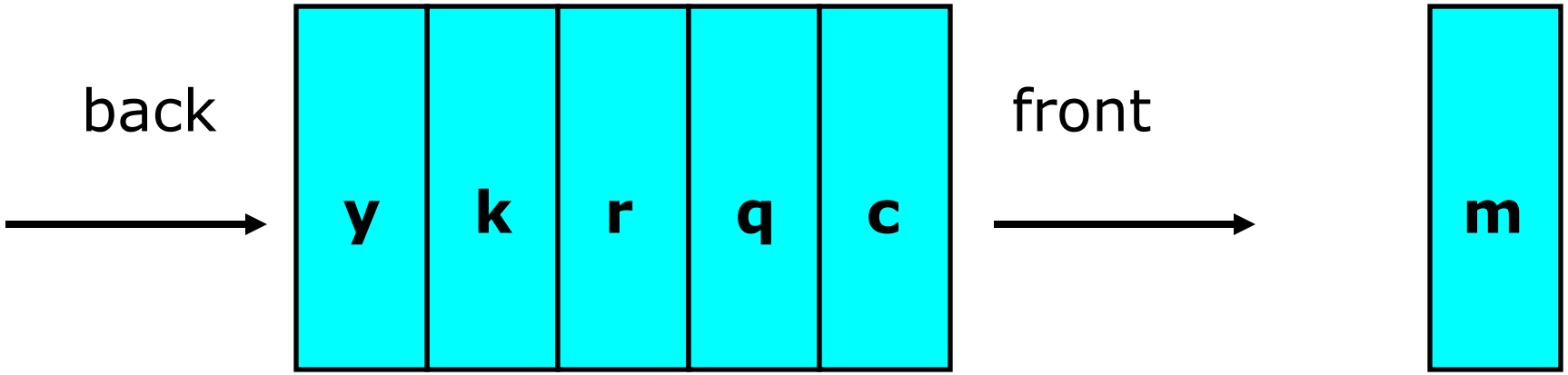
Queues are FIFO

Enqueue operation:



Queues are FIFO

Dequeue operation:



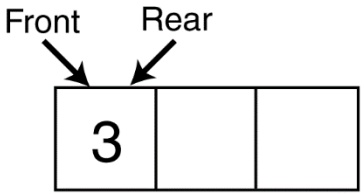
Queue Operations

Suppose we have an empty static integer queue that is capable of holding a maximum of three values. With that queue we execute the following enqueue operations.

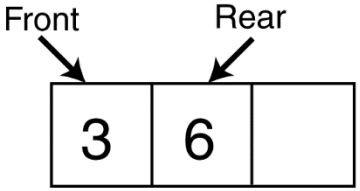
```
Enqueue(3);
Enqueue(6);
Enqueue(9);
```

The queue after each of the enqueue operations.

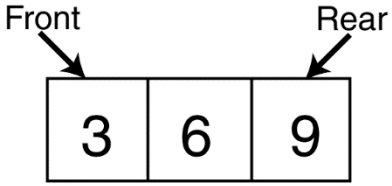
Enqueue(3);



Enqueue(6);

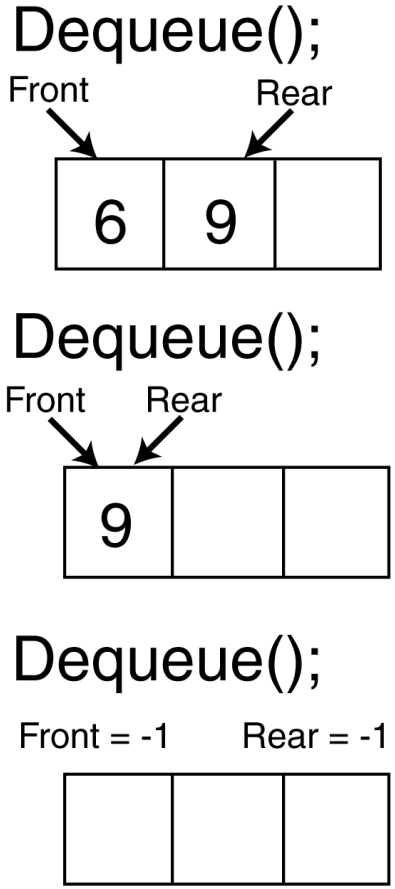


Enqueue(9);



Queue Operations

The queue after each of three consecutive dequeue operations



Queue initialization

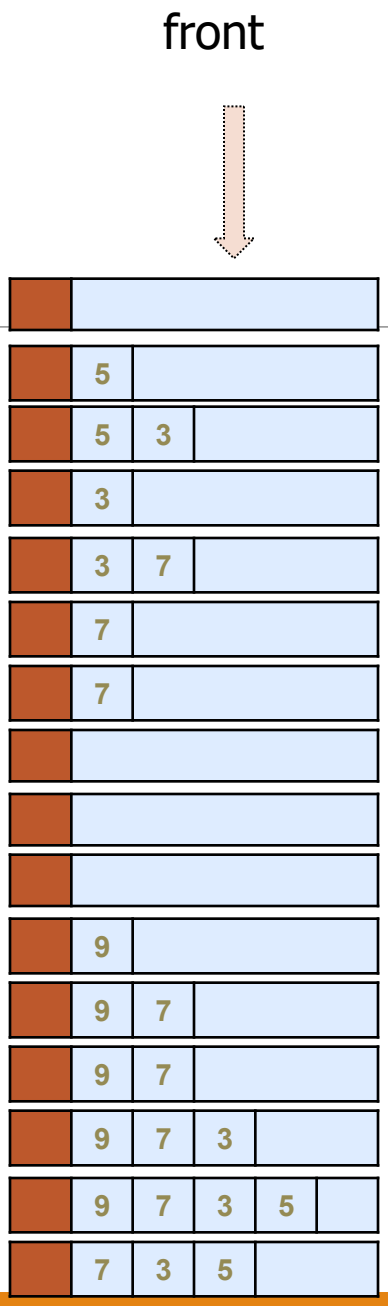
For an empty queue



Front, rear For an empty queue rear must be initialized to front-1, rear=-1 and size=0

Queue Example

Operation	Output
enqueue(5)	-
enqueue(3)	-
dequeue()	5
enqueue(7)	-
dequeue()	3
front()	7
dequeue()	7
dequeue()	"error"
isEmpty()	true
enqueue(9)	-
enqueue(7)	-
size()	2
enqueue(3)	-
enqueue(5)	-
dequeue()	9



Implementing a Queue

Implementation is also based on an array with restricted access.

It would be accessed as a queue would be.

Elements in the array would be inserted at the back and removed from the front.

peek()

Gets the element at the front of the queue without removing it.

Algorithm

```
begin procedure peek
return queue[front]
end procedure
```

isfull()

As we are using single dimension array to implement queue, we just check for the rear pointer to reach at MAXSIZE to determine that the queue is full.

```
begin procedure isfull

   if rear equals to MAXSIZE -1
      return true
   else
      return false
   endif

end procedure
```

isempty()

```
begin procedure isempty

  if front is less than MIN  OR front is greater than rear
    return true
  else
    return false
  endif

end procedure
```

Enqueue Operations

Step 1 − Check if the queue is full.

Step 2 − If the queue is full, produce overflow error and exit.

Step 3 − If the queue is not full, increment rear and front pointer to point the next empty space.

Step 4 − Add data element to the queue location, where the rear is pointing.

Step 5 − return success.

Algorithm for enqueue operation

```
procedure enqueue(data)
  if queue is full
    return overflow
  endif

  rear ← rear + 1

  queue[rear] ← data

  return true

end procedure
```

Dequeue Operation

- **Step 1** – Check if the queue is empty.
- **Step 2** – If the queue is empty, produce underflow error and exit.
- **Step 3** – If the queue is not empty, access the data where **front** is pointing.
- **Step 4** – Increment **front** pointer to point to the next available data element.
- **Step 5** – Return success.

Algorithm for dequeue operation

```
procedure dequeue
   if queue is empty
      return underflow
   end if

   data = queue[front]
   front ← front + 1

   return true
end procedure
```

Exercise 1

Perform following operations in a queue where the size of the queue is 6.

Enqueue 5 ,4,7

Dequeue

Dequeue

Enqueue 8,9

Dequeue

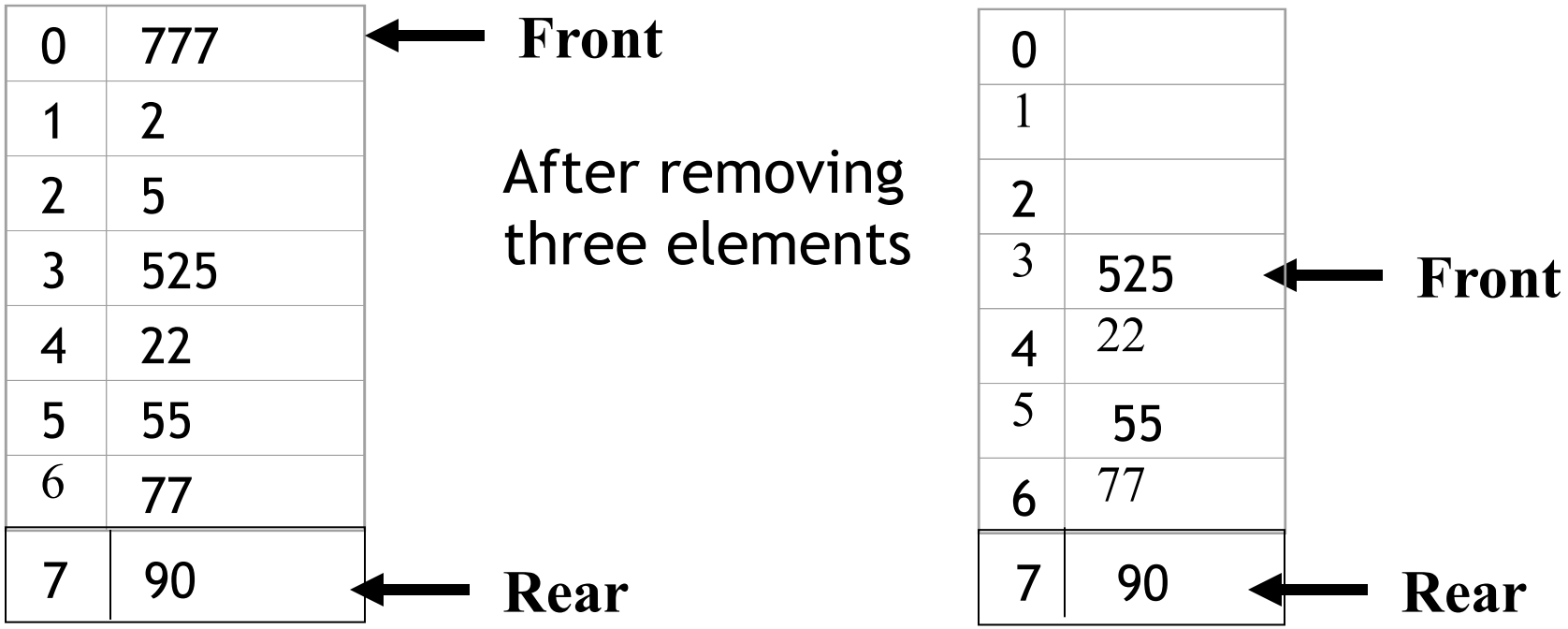
Enqueue 1,2

Attention .....!!!!

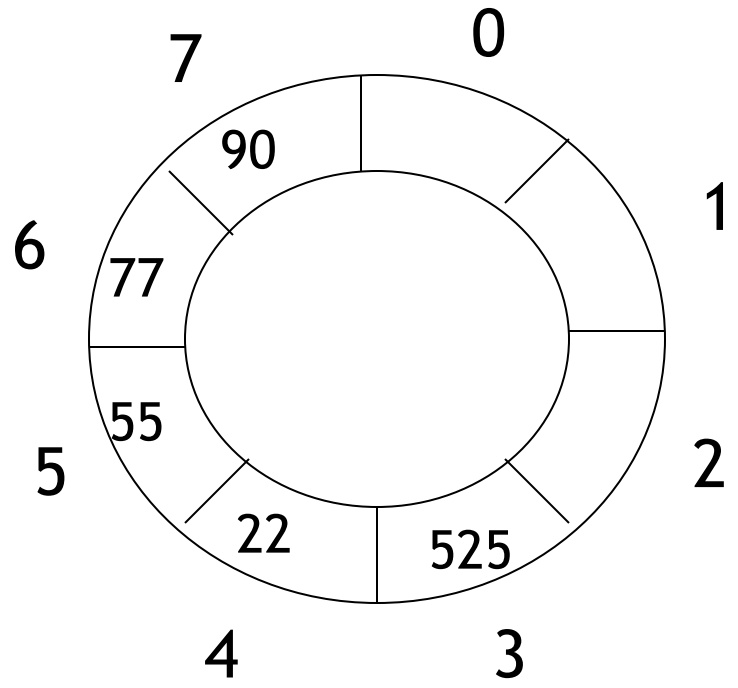
Up to now we discussed operations on Queues which are linear, where when the items are removed the front is increased and the size of the queue is reduced.

For example if we have the following queue

Although the queue is not full we cannot insert more elements.

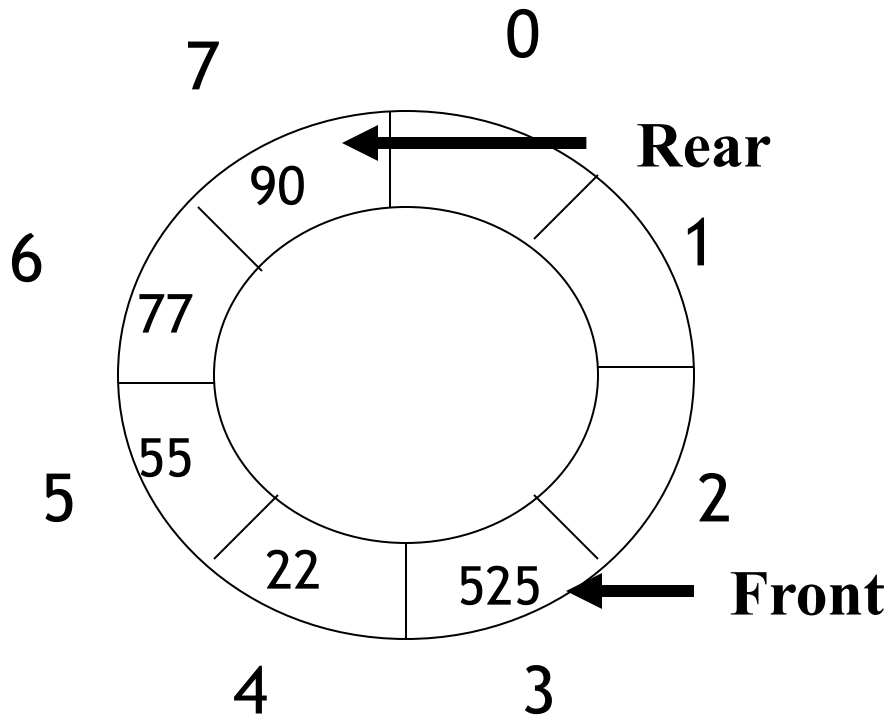


How to overcome this problem …..????
We can use a circular queue



0	
1	
2	
3	525
4	22
5	55
6	77
7	90

Circular Queue



0	
1	
2	
3	525
4	22
5	55
6	77
7	90

Circular Queues

Circular queues are queues that wrap around themselves. These are also called ring buffers.

The problem in using the linear queue can be overcome as follows.

When we want to insert a new element, we can insert it at the beginning of the queue.

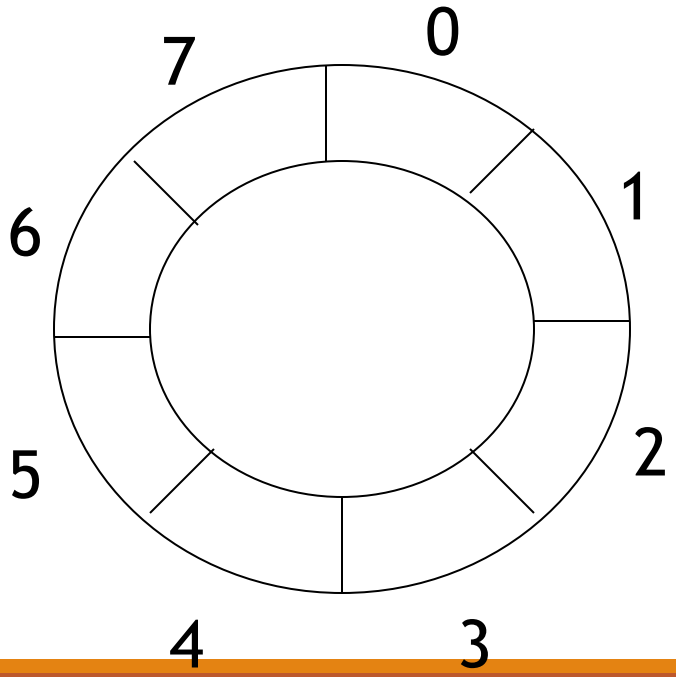
- i.e. if the queue is not full we can make the rear start from the beginning by wrapping around.

Circular Queue

If rear was 2 then the next element should be stored in index 3

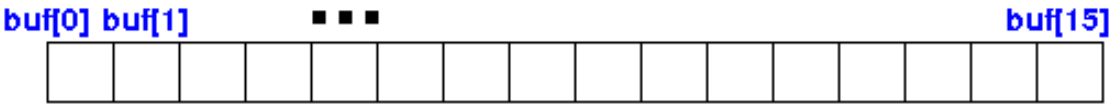
If rear was 4 then the next element should be stored in index 5

If rear was 7 then the next element should be stored in index 0

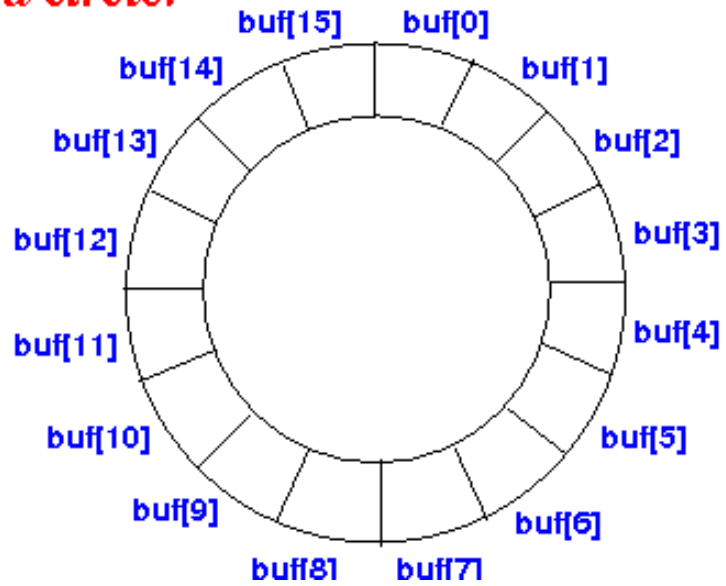


Implementing a Queue using a circular array

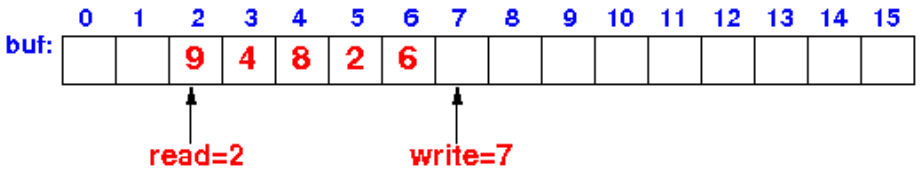
Array:

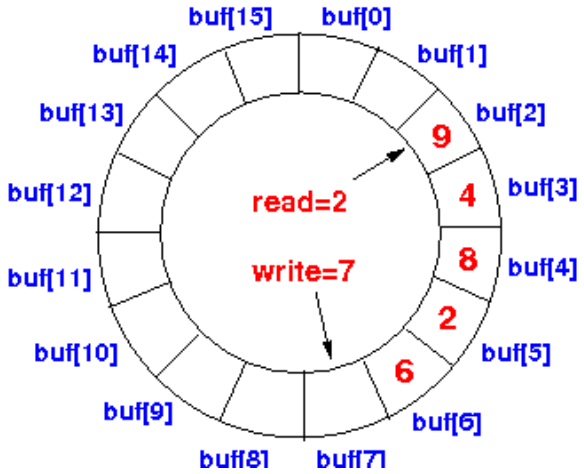


Pretend array is a circle:



Implementing a Queue using a circular array





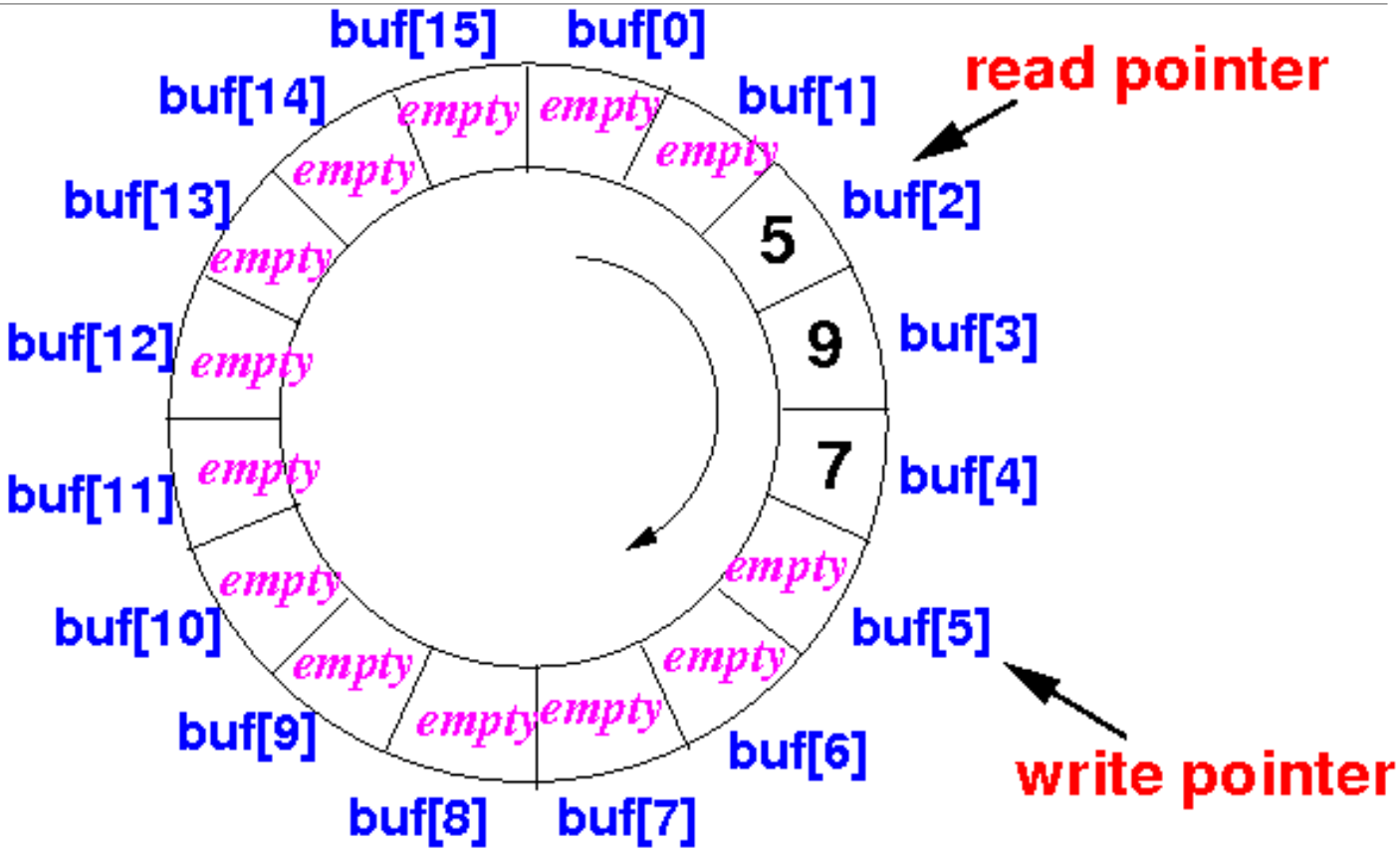
Read pointer = the **index** of the **circular array** used by a ***read* operation**

Example:

In the **above figure**, a ***read* operation** will return the **value 9** in **buf[2]** (because the ***read* pointer read = 2**)

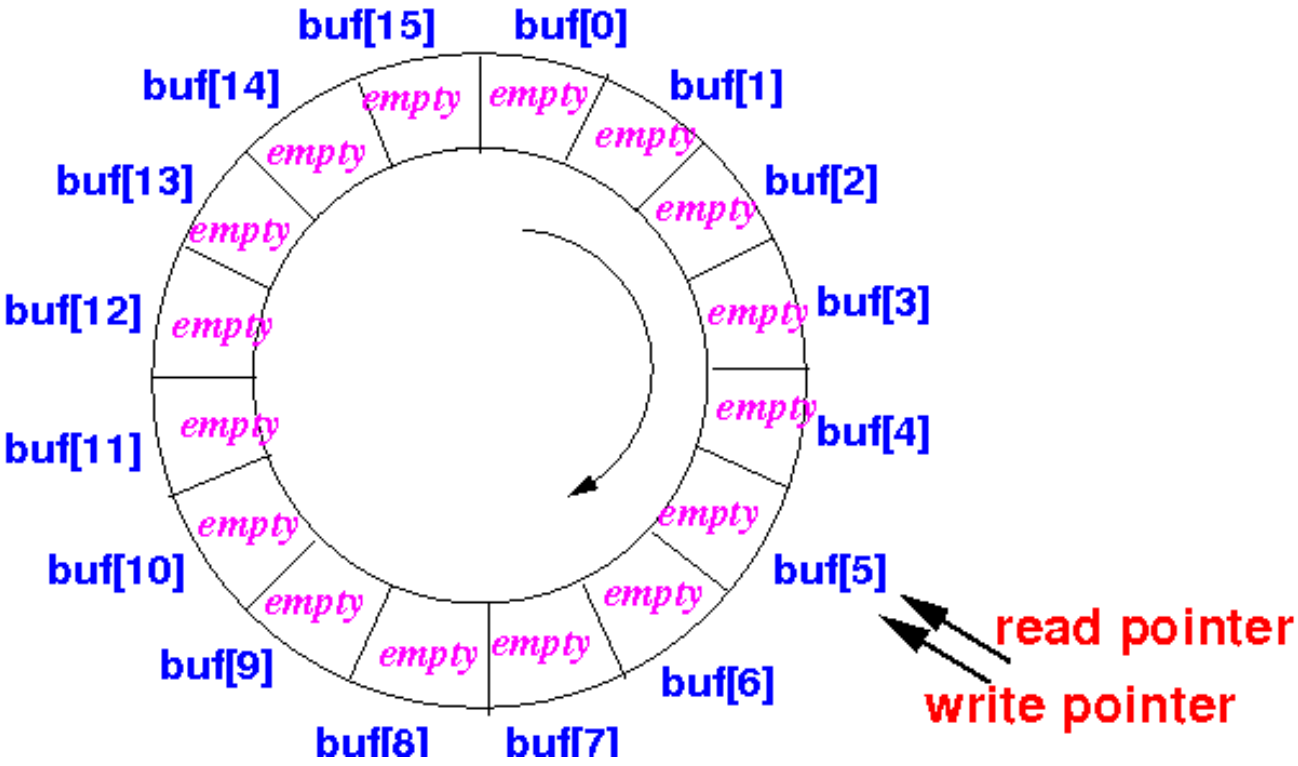
Implementing a Queue using a circular array

circular buffer (array) with **3 data items**:



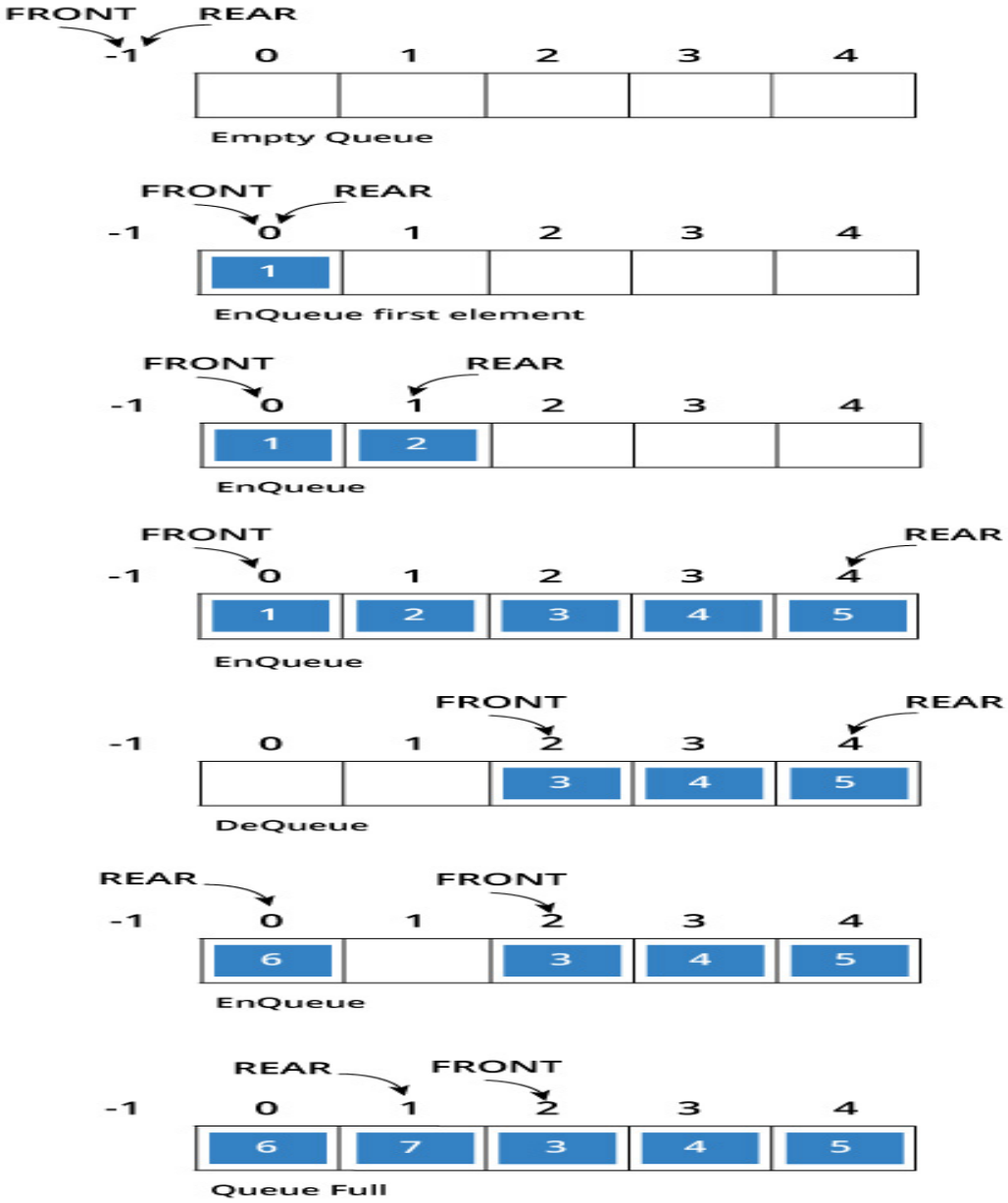
Implementing a Queue using a circular array

After **reading (removing) *two* more data item**, the **circular buffer (array)** will become ***empty***:



a **circular array (buffer)** is ***empty*** if:

read pointer == write pointer


FRONT
REAR
-1
0
1
2
3
4
Empty Queue
FRONT
REAR
1
EnQueue first element
FRONT
REAR
1
2
EnQueue
FRONT
REAR
1
2
3
4
5
EnQueue
FRONT
REAR
3
4
5
DeQueue
REAR
FRONT
6
3
4
5
EnQueue
REAR
FRONT
6
7
3
4
5
Queue Full

Implementation of Circular Queue

- **Step 1:** Include all the **header files** which are used in the program and define a constant **'SIZE'** with specific value.
- **Step 2:** Declare all **user defined functions** used in circular queue implementation.
- **Step 3:** Create a one dimensional array with above defined SIZE (**int cQueue[SIZE]**)
- **Step 4:** Define two integer variables **'front'** and '**rear**' and initialize both with **'-1'**. (**int front = -1, rear = -1**)
- **Step 5:** Implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on circular queue.

enQueue(value) - Inserting value into the Circular Queue

- **Step 1:** Check whether queue is FULL. ((rear == SIZE-1 && front == 0) || (front == rear+1))
- **Step 2:** If it is FULL, then display "Queue is FULL!!! Insertion is not possible!!!" and terminate the function.
- **Step 3:** If it is NOT FULL, then check rear == SIZE - 1 && front != 0 if it is TRUE, then set rear = -1.
- **Step 4:** Increment rear value by one (rear++), set queue[rear] = value and check 'front == -1' if it is TRUE, then set front = 0.

Exercise 2

Perform following operations in a circular queue where the size of the queue is 4.

Enqueue 5 ,4,7

Dequeue

Enqueue 8,9

Dequeue

Enqueue 1,2

END